RV College of Engineering®

UNIT 3

Cycle GAN

Prof. Somesh Nandi
Assistant Professor
Dept. of AI and ML
RVCE, Bangalore

Go, change the world®

A GAN is a battle between two adversaries, the *generator* and the *discriminator*.

The generator tries to convert random noise into observations that look as if they have been sampled from the original dataset, and the discriminator tries to predict whether an observation comes from the original dataset or is one of the generator's forgeries.

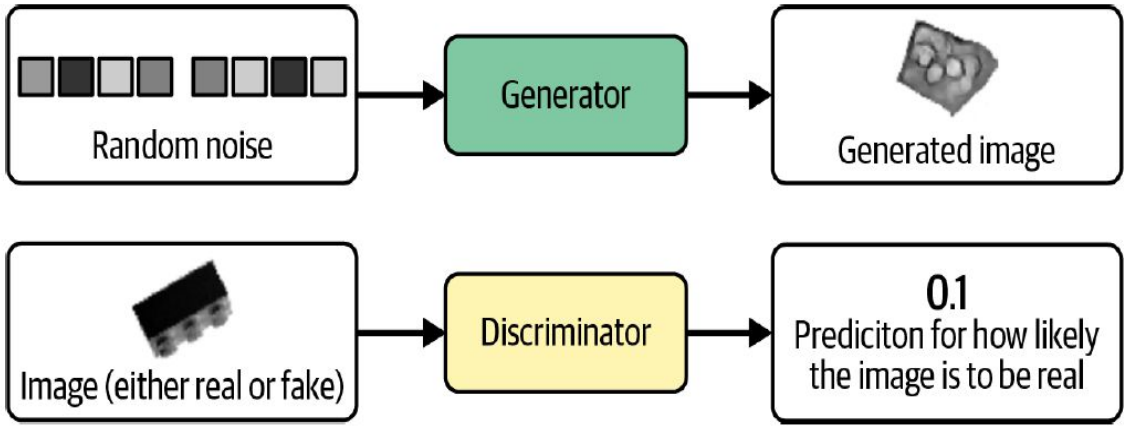
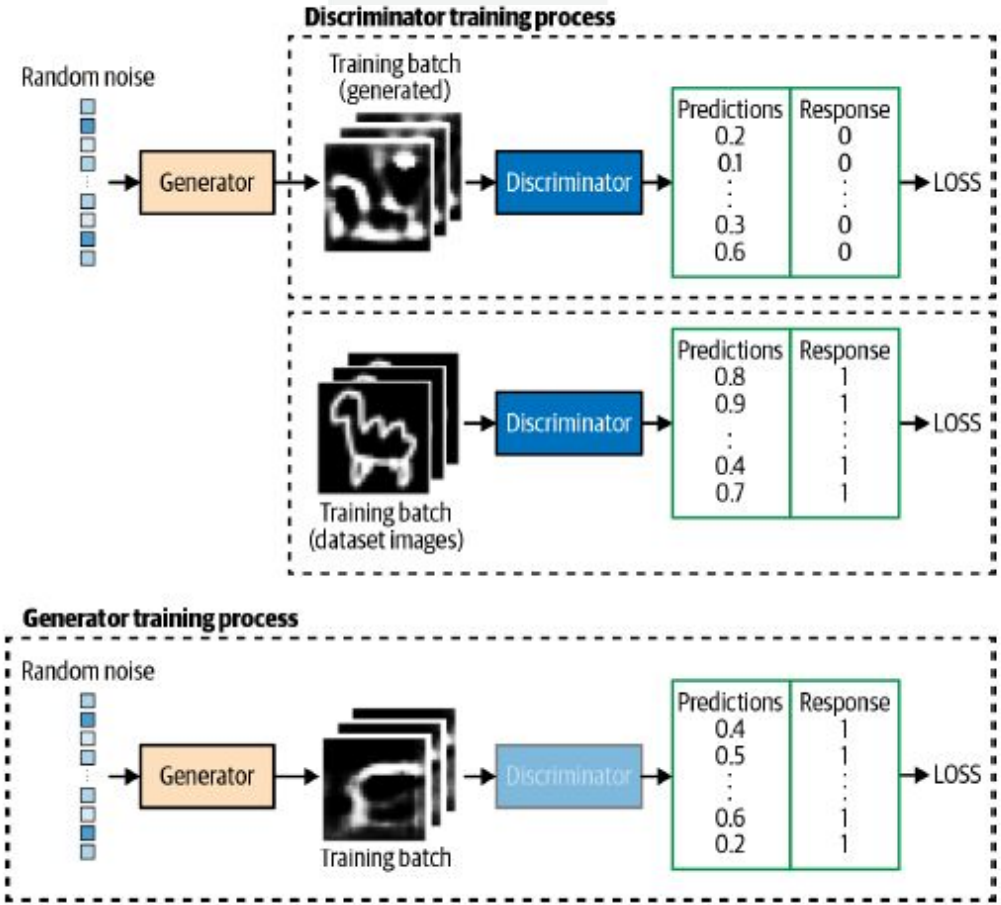


Figure 4-2. Inputs and outputs of the two networks in a GAN

GAN


Discriminator training process
Random noise
Generator
Training batch (generated)
Discriminator
Predictions
0.2
0.1
0.3
0.6
Response
0
0
0
0
LOSS
Discriminator
Predictions
0.8
0.9
0.4
0.7
Response
1
1
1
1
LOSS
Training batch (dataset images)
Generator training process
Random noise
Generator
Training batch
Discriminator
Predictions
0.4
0.5
0.6
0.2
Response
1
1
1
1
LOSS

- Generator = **student forging a signature**
- Discriminator = **teacher checking if it's fake**

Over time:

- Student gets better at copying
- Teacher gets better at spotting

Final Outcome

Eventually:

- Generator creates **very realistic images**
- Discriminator finds it **hard to tell real vs fake**

GAN Challenges

- Oscillating Loss
- Mode Collapse
- Uninformative Loss
- Hyper parameter

1. Oscillating Loss

Instead of steadily decreasing, the loss values of the generator and discriminator keep fluctuating.

Why it happens:

- GAN training is like a game (minimax optimization), not simple minimization.
- When one network improves, the other reacts → continuous back-and-forth.

Effect:

- No clear convergence.
- Training may never stabilize.

Example:

- Generator improves → discriminator gets worse → discriminator adapts → generator struggles again.

1. Oscillating Loss

Imagine training a GAN to generate handwritten digits (like **MNIST dataset**).

Initially: Generator produces poor digits → Discriminator easily detects fake → **Generator loss high**

After some training: Generator improves → Discriminator gets confused → **Discriminator loss increases**

Then: Discriminator adapts → becomes strong again → Generator struggles

Result:
Loss graph looks like a zig-zag (up and down), not smooth.

2. Mode Collapse

The generator produces very limited or identical outputs regardless of input. Mode collapse occurs when the generator finds a small number of samples that fool the discriminator and therefore isn't able to produce any examples other than this limited set.

Why it happens:

- Generator finds a few outputs that consistently fool the discriminator.
- Instead of learning the full data distribution, it "cheats" with repetition.

Effect:

- Lack of diversity in generated data.
- Example: GAN generates the same face or same digit repeatedly.

2. Mode Collapse

Again using **MNIST dataset**:

Instead of generating digits 0–9,

Generator produces only **“3”** (or a few similar digits)

Even if you input different noise:

Output = same digit “3” again and again

Another example:

Face generation GAN outputs the **same face with slight variations**

Result:

No diversity in outputs

3. Uninformative Loss

Loss values don’t reflect actual performance or learning progress.

Why it happens:

- GAN loss functions (like binary cross-entropy) don’t correlate well with output quality.
- Even when images improve, loss might not change meaningfully.

Effect:

- Difficult to monitor training.
- Hard to decide when to stop or tune the model.

3. Uninformative Loss

Suppose you are training a GAN to generate human faces.

Loss values: Stay around **0.5 or constant**

But visually: Images improve significantly over time

OR

Loss decreases But generated images are still poor

👉 **Result:**

Loss does NOT match actual performance.

Example case:

Using standard GAN loss (binary cross-entropy)

Discriminator becomes too strong → gradients vanish → loss stops being meaningful

4. Hyperparameter Sensitivity

GANs are extremely sensitive to hyperparameters.

Examples of hyperparameters:

- Learning rate
- Batch size
- Network architecture
- Optimizer (Adam, RMSProp)
- Training ratio (generator vs discriminator updates)

Why it matters:

Small changes can lead to:

- Training failure
- Mode collapse
- Divergence

Effect:

- Requires careful tuning and experimentation.

4. Hyper parameter Sensitivity Learning Rate

Learning rate too high: Generator produces random noise Training becomes unstable

Learning rate too low: Training is extremely slow or stuck

- Need paired data
  No mapping consistency

Challenge	Problem	Result
Oscillating Loss	Loss keeps fluctuating	No convergence
Mode Collapse	Limited output diversity	Repetitive results
Uninformative Loss	Loss doesn’t reflect quality	Hard to evaluate training
Hyperparameters	Highly sensitive settings	Instability or poor training

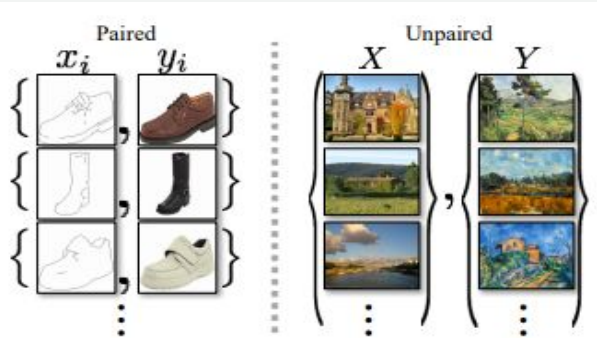
Cycle GAN

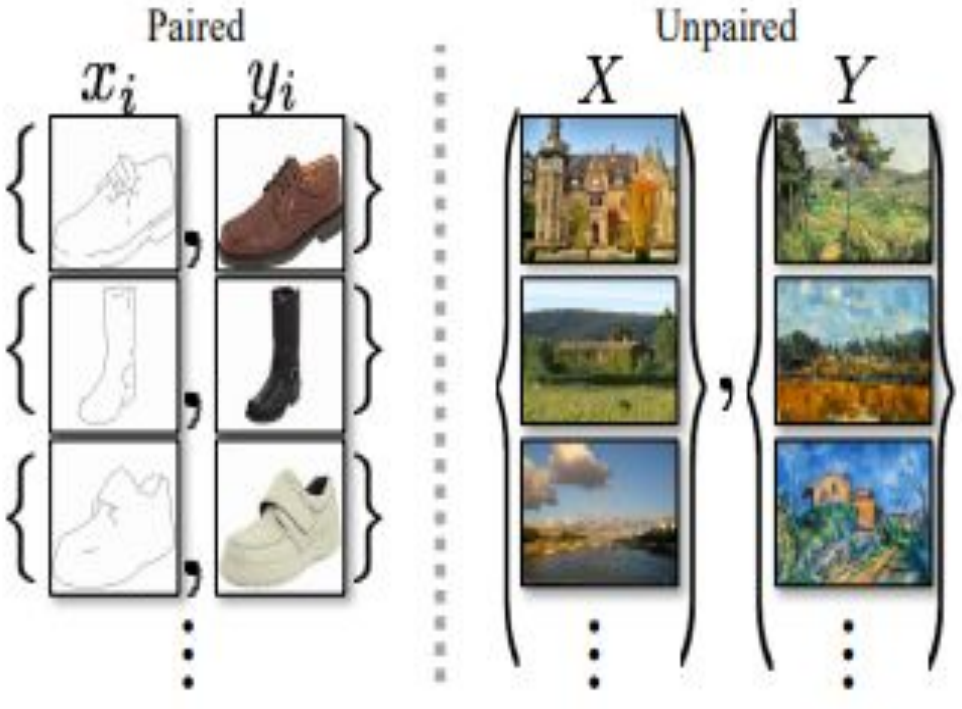
A **CycleGAN** (Cycle-Consistent Generative Adversarial Network) is a type of deep learning model used for **image-to-image translation** without needing **paired training data**.

It was introduced in the paper **"Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks"** by Jun-Yan Zhu et al., in 2017.

CycleGAN learns to **translate images from one domain to another** (e.g., horses ↔ zebras, summer ↔ winter scenes, or paintings **having corresponding image pairs**.

GAN = “Make it look real”
CycleGAN = “Make it look real AND stay related to input”





Core Idea of CycleGAN

“If I convert A → B, I should be able to convert it back B → A”

This is called **cycle consistency**

Cycle GAN

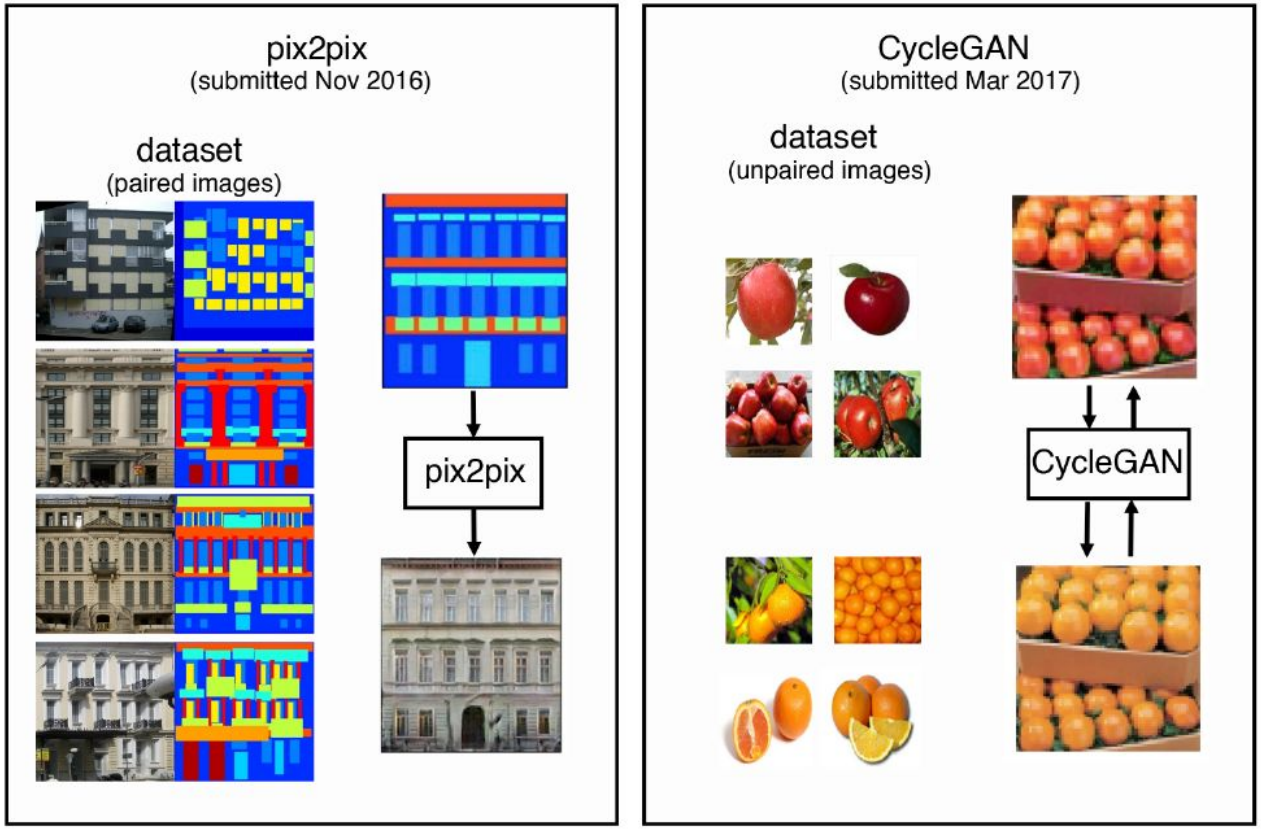


Figure 5-4. pix2pix dataset and domain mapping example

Cycle GAN

A CycleGAN is actually composed of four models, two generators and two discriminators.

- The first generator, G_AB, converts images from domain A into domain B.
- The second generator, G_BA, converts images from domain B into domain A.

•Generators:

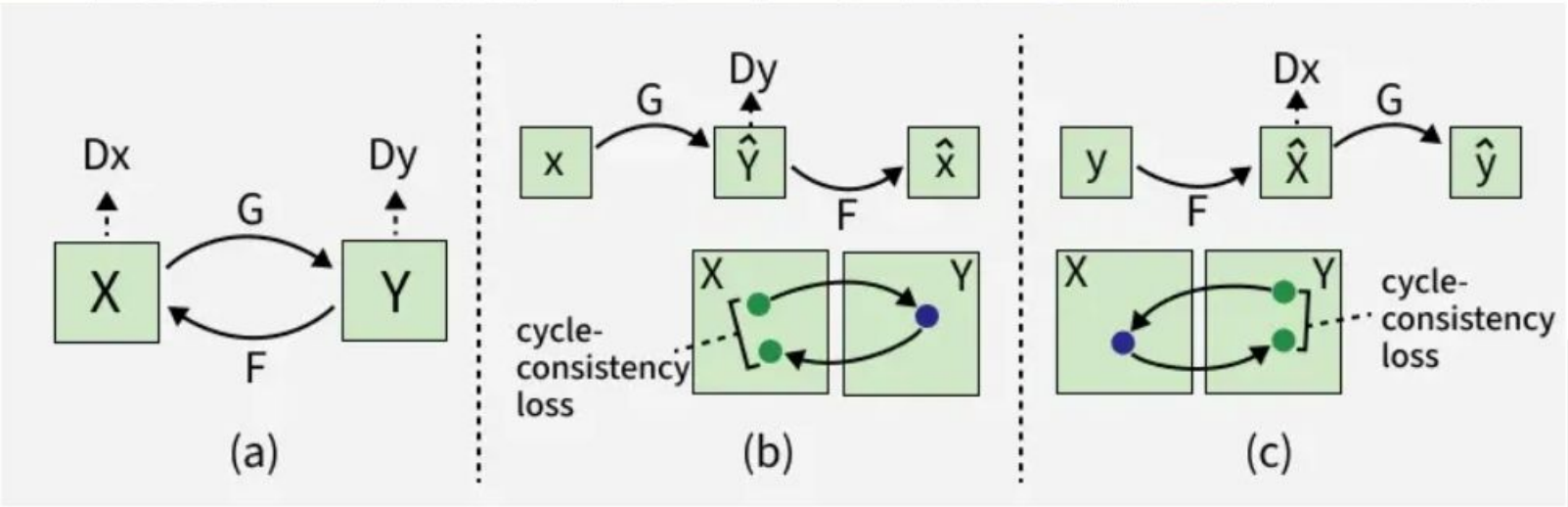
- G:X→Y (translates images from domain X to Y)
- F:Y→X (translates images from domain Y back to X)

•Discriminators:

- Dy: distinguishes real Y images from fake ones generated by G
- Dx: distinguishes real X images from fake ones generated by F

- The process starts with an input image(x) and Generator G translates it to the target domain like turning a photo into a painting. Then generator F takes this transformed image and maps it back to the original domain helps in reconstructing an image close to the input.

- The model measures the difference between the original and reconstructed images using a loss function like mean squared error. This cycle consistency loss helps the network to learn meaningful, reversible mappings between the two domains.


Dx
Dy
G
X
Y
F
(a)
G
Dy
x
ŷ
x̂
F
X
Y
cycle-consistency loss
(b)
Dx
G
y
x̂
ŷ
F
X
Y
cycle-consistency loss
(c)

Cycle GAN

- **G** → converts **X** → **Y** (Horse → Zebra)
- **F** → converts **Y** → **X** (Zebra → Horse)
- **D_x (Dx)** → checks if something is a real **X (horse)**
- **D_Y (Dy)** → checks if something is a real **Y (zebra)**

Cycle GAN

- **(b) Middle Part – First Cycle Step-by-step:**
- Take an image from **X (horse)**
- Pass through **G**
  → becomes **fake Y (zebra)**
- Pass that into **F**
  → becomes **reconstructed X (horse)**

Key idea here:

- Original X ≈ Reconstructed X

Cycle GAN

- Same idea but opposite direction:
- Start with **Y (zebra)**
- Pass through **F** → fake X (horse)
- Pass through **G** → reconstructed Y (zebra)

Again:

- Original Y ≈ Reconstructed Y

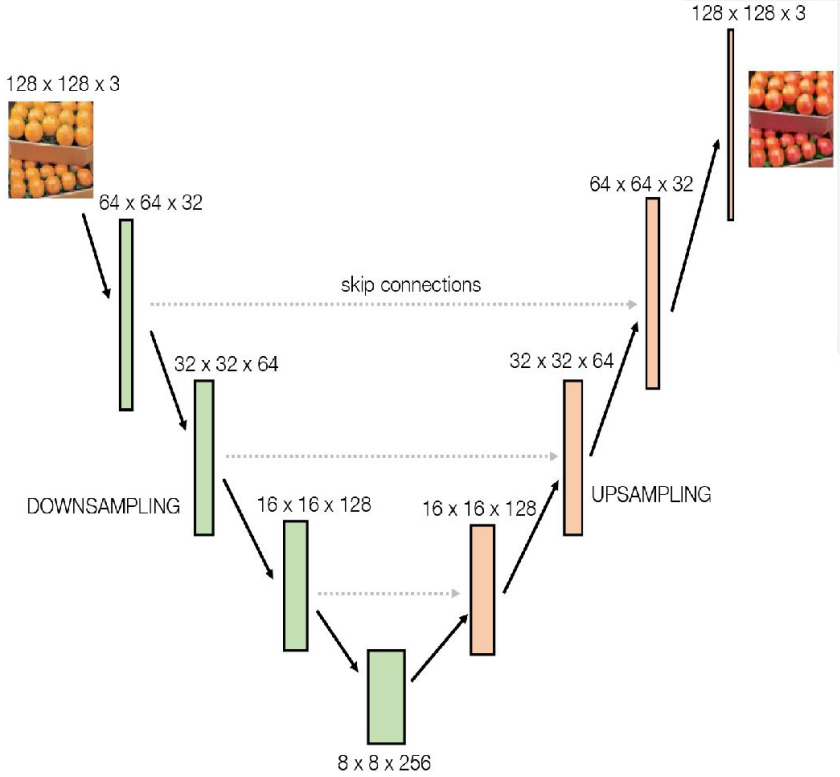
Convert it, convert it back—if you don’t get the same thing, you did it wrong.

Cycle GAN

Typically,
CycleGAN generators take one of two forms: ***U-Net* or *ResNet*** (residual network).

Cycle GAN

The Generators U-Net



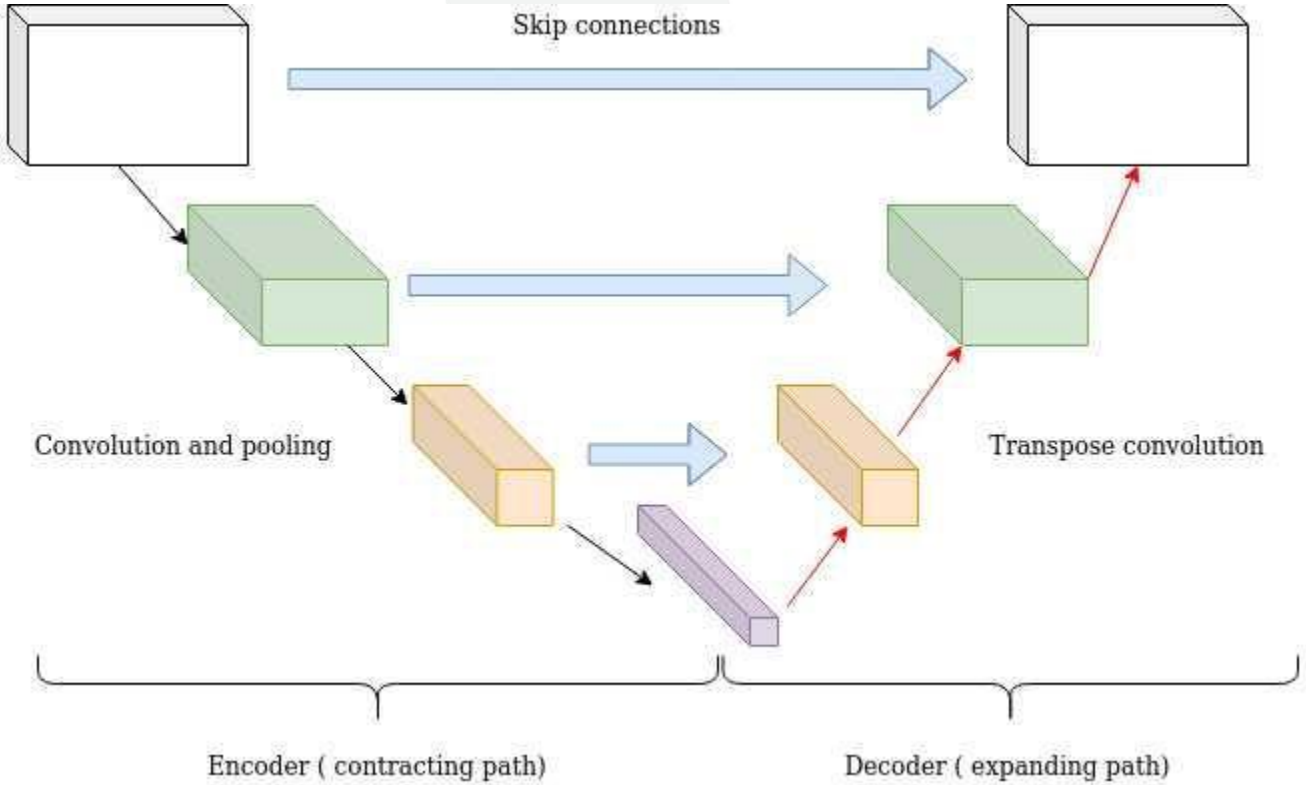
U-Net consists of two halves:

Downsampling half, where input images are compressed spatially but expanded channel-wise.

- Convolution layers with **stride > 1**
- Sometimes **max pooling**

Upsampling half, where representations are expanded spatially while the number of channels is reduced.

- Transposed convolution (ConvTranspose)
- Interpolation (nearest/bilinear) + convolution


Skip connections
Convolution and pooling
Transpose convolution
Encoder (contracting path)
Decoder (expanding path)

- **STEP 1: Input Image**
- You give an image to the model.

Example:

- A street image with cars, trees, people

STEP 2: Downsampling (Encoder – “Understand”)

- Now the model starts **zooming out**:
- Image becomes smaller
- But it learns more features
- What happens:
- Learns: “There is a car”
- But loses: “exact location of the car”
- Think:
- Like looking at Google Maps zoomed out

- **STEP 3: Deep Understanding (Bottom of U)**
- At the center:
- Model has maximum understanding
- Minimum detail
- It knows:
- “This is a car scene”
- But:
- Not precise boundaries

STEP 4: Upsampling (Decoder – “Rebuild”)

- Now the model starts **zooming back in**:
- Image becomes larger again
- Tries to reconstruct original size
- Problem:
- It forgot exact details

Skip Connections

- Takes **details from downsampling**
- Combines them with **upsampling**
- 💡 Example:
- Upsampling says: “There is a car”
- Skip connection says: “It is HERE”
- 👉 Combine → Accurate result

To build in the skip connections, we will need to introduce a new type of layer: Concatenate

Imagine:

- Earlier layer (from downsampling):
  "I know exact positions (details)"
- Current layer (upsampling):
  "I know what the object is"
- Now we do:

Concatenate([current_layer, earlier_layer])

- Result:
- One layer that knows **both WHAT and WHERE**

The Discriminators

Old Way (Normal GAN)

- Discriminator gives **ONE output**

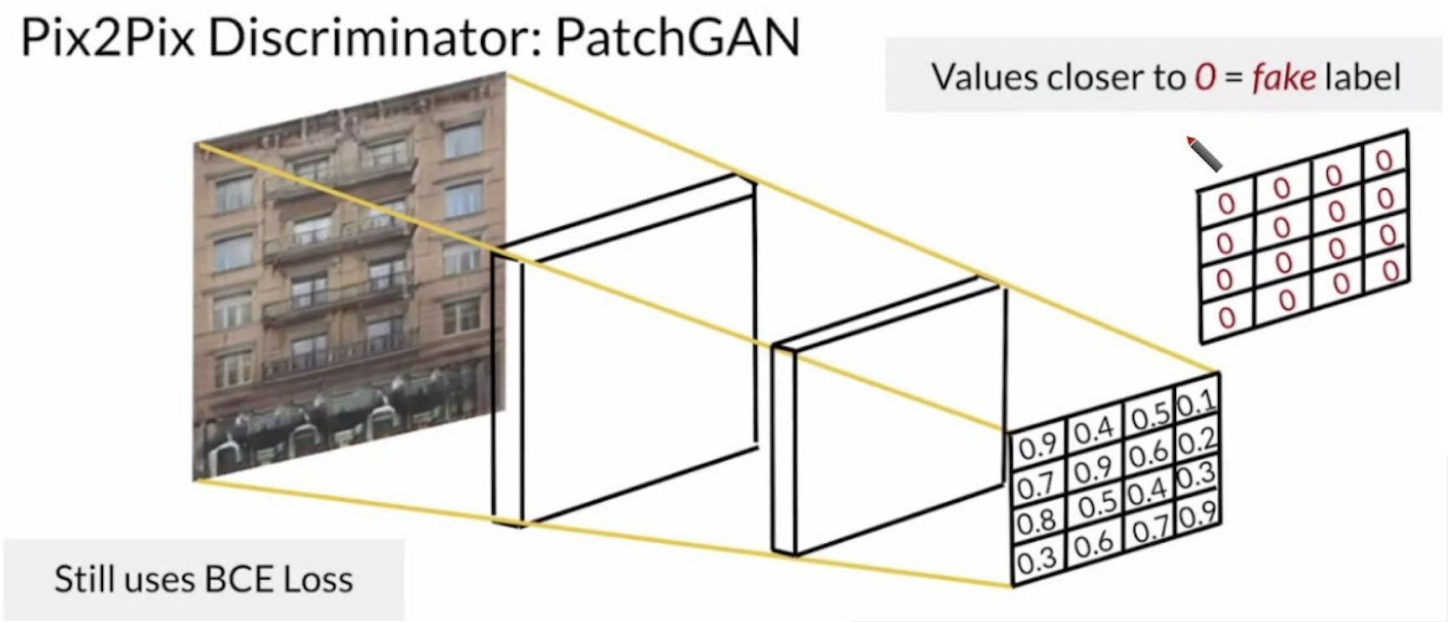
Example:

- Input: Image
- Output: 0.9 → “Looks real”
- Output: 0.1 → “Looks fake”

Problem:

- It judges the **whole image at once**

CycleGAN Way (PatchGAN)



"Let me check **small parts (patches)** of the image!"

Cycle GAN Way (Patch GAN)

What is a Patch?

A patch = small square part of the image

Example:

- Image → divided into many small squares

Like:

```
[ □ □ □ □ ]
[ □ □ □ □ ]
[ □ □ □ □ ]
[ □ □ □ □ ]
```

Instead of:

```
0.8
```

You get:

```
8 x 8 grid
```

Example:

```
[0.9 0.8 0.7 ...]
[0.6 0.5 0.9 ...]
...
```

Whole Image Check

- Focuses on **content** (e.g., “Is this a horse?”)

PatchGAN

- Focuses on **style/texture** (e.g., “Do the stripes look real?”)

Cycle GAN Way (Patch GAN)

Step	Size	Meaning
Input	256×256	Original image
Layer 1	128×128	Basic features
Layer 2	64×64	More features
Layer 3	32×32	Deeper features
Layer 4	32×32	Refined
Output	~8×8	Patch predictions

- Looks at a **small patch of image**
  So instead of:
- "Is whole image real?"
  It asks:
- "Is THIS patch real?"

RV College of Engineering®

Compiling the Cycle GAN

Go, change the world®

CycleGAN is not just one model—it's a **team of 4 models working together**.

- It uses **two generators**:
- **g_AB** → converts A → B
- **g_BA** → converts B → A
- It uses **two discriminators**:
- **d_A** → checks if images are real apples or fake
- **d_B** → checks if images are real oranges or fake
- Generators try to **fool the discriminators**, while discriminators try to **detect fake images**.
- CycleGAN works between **two domains**: A (apples) and B (oranges).
- Discriminators can be compiled directly because they simply **classify images as real (1) or fake (0)**.

Compiling the Cycle GAN

Compiling the discriminator

```
self.d_A = self.build_discriminator()
self.d_B = self.build_discriminator()
self.d_A.compile(loss='mse',
optimizer=Adam(self.learning_rate, 0.5),
metrics=['accuracy'])
self.d_B.compile(loss='mse',
optimizer=Adam(self.learning_rate, 0.5),
metrics=['accuracy’])
```

But How do I Compile the Generators?

Compiling the Cycle GAN

Compiling the discriminator

1. Validity (Looks Real?)
The generated image should **fool the discriminator**
Apple → Orange should look like a real orange
If it looks fake → generator is bad

2. Reconstruction (Cycle Check)
Convert and convert back:
Apple → Orange → Apple
You should get the **same original apple**
If not → model is losing important information

3. Identity (Don't Change What's Already Correct)
If you give:
Orange → generator (apple→orange)
It should **stay orange (no unnecessary change)**

RV College of Engineering®

Write a Python program using Keras to build and compile the combined CycleGAN model.

Go, change the world®

- Your implementation should:
- Create two generators:
  - One for domain A → B
  - One for domain B → A
- Use two discriminators (assume they are already defined)
- Freeze the discriminators during generator training
- Take two inputs: images from domain A and domain B
- Generate:
  - Fake images in both domains
  - Reconstructed images (cycle consistency)
  - Identity mappings
- Pass generated images through discriminators to check validity
- Build a combined model that outputs:
  - Validity (real/fake predictions)
  - Reconstruction outputs
  - Identity outputs
- Compile the model with:

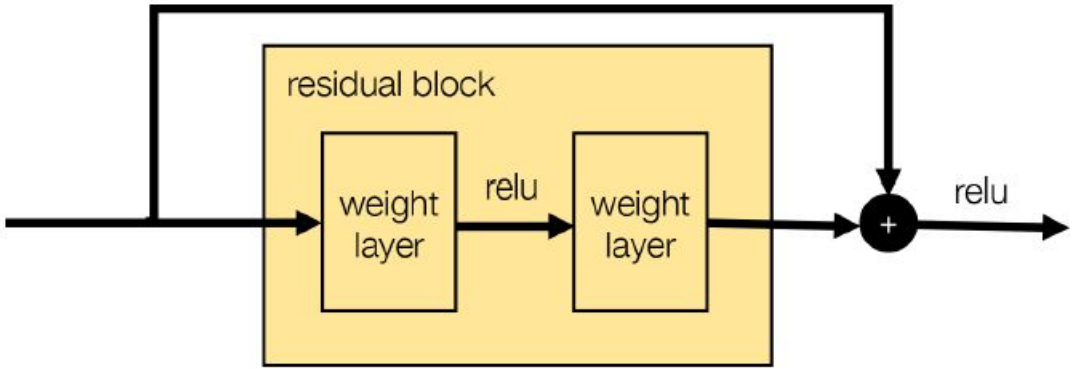
The Generators (ResNet)

- Think of a neural network like a **factory assembly line** that keeps modifying an image step by step.
- Normally:
- Each layer **changes the image a bit more**
- But as the network becomes very deep, it starts to **forget earlier information**

- ResNet solves this by adding shortcuts

The Generators (ResNet)

- There are **two paths**:
- **1. Main path (processing path)**
- Input → weight layer → ReLU → weight layer
- **2. Shortcut path (top line)**
- Input goes **directly to the end** without changes



Step 1: Input comes in

•This is your original data (image/features)

Step 2: Main path processes it

Inside the yellow box:

1. **Weight layer** → extracts features
2. **ReLU** → adds non-linearity (helps learning complex patterns)
3. **Weight layer** → further refines features

This produces: **"changes" (or improvements)**

Step 3: Shortcut skips everything

•The same input is sent **directly to the end**

•No processing, no modification

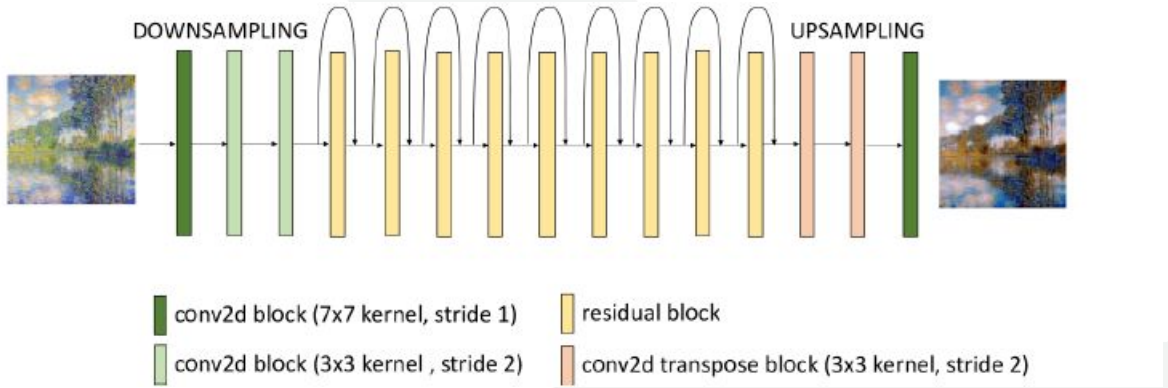
Step 4: Addition happens (+)

At the circle:

```
Final Output = Original Input + Processed Output
```

This is the **core idea of ResNet**

ResNet generator



ResNet generator

Problem: *Vanishing Gradient*

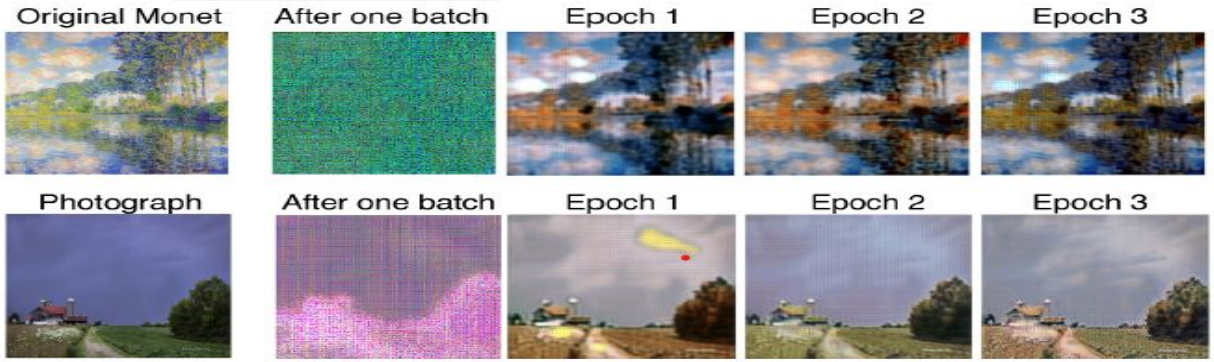
- As the signal goes backward through many layers, it becomes **very small**
- Early layers (near input) **almost stop learning**
- Result:
- Deep networks become **slow or useless to train**

What ResNet does differently

- ResNet introduces **skip connections (shortcuts)**
- Instead of forcing gradients to pass through every layer:
- It gives them a **direct highway**

Analysis of the Cycle GAN

- At first, the model produces **bad, unrealistic images**
  With training, it **gradually improves**
  Until it can **convincingly switch styles both ways**



Direction	What happens
Painting → Photo	Removes artistic style
Photo → Painting	Adds artistic style

Neural Style Transfer

No dataset needed

- Only **2 images**:
  - **Content image** → what you want to keep
  - **Style image** → how you want it to look
- Goal:
- Mix them into **one final image**

Example

- Content image: your photo 📷
- Style image: a Monet painting 🎨

Output:

- Your photo **painted like Monet**

Loss Function

The model uses **3 goals at the same time**:

Content Loss

- "Don't change what's in the image"
- Keep:
  - Objects
  - Shapes
  - Structure

Example:

- If it's a photo of a dog
  → Output should still be a dog

Loss Function

Style Loss

- “Make it look like the style image”
- Add:
  - Colors
  - Brush strokes
  - Texture

Example:

- Make dog look like a painting

Loss Function

Total Variation Loss

- “Make the image smooth”
- Removes:
  - Noise
  - Pixelated artifacts

Think:

- Avoid rough, noisy output → make it clean

- **Final Goal**
- The model tries to balance all three:
- Final Loss = Content + Style + Smoothness

Loss Function

Step-by-step idea

- Start with a random image (or content image)
- Check:
  - How different is it from content?
  - How different is it from style?
- Adjust pixels slightly
- Repeat many times

This is called Gradient Descent

- Simple meaning:
- “Make small improvements again and again”
- Each step → image gets better
- Loss decreases gradually

Content Loss

Does this new image still contain the same main objects and structure as the original image?”

For example:

- Original image → a city with buildings, river, and sky
- Generated image → maybe painted in Van Gogh style

- Even after adding the artistic style, we still want:
- buildings to remain buildings
- river to remain river
- sky to remain sky
- objects to stay roughly in the same positions

Content Loss

- Take a neural network that already knows how to recognize images (like VGG16).

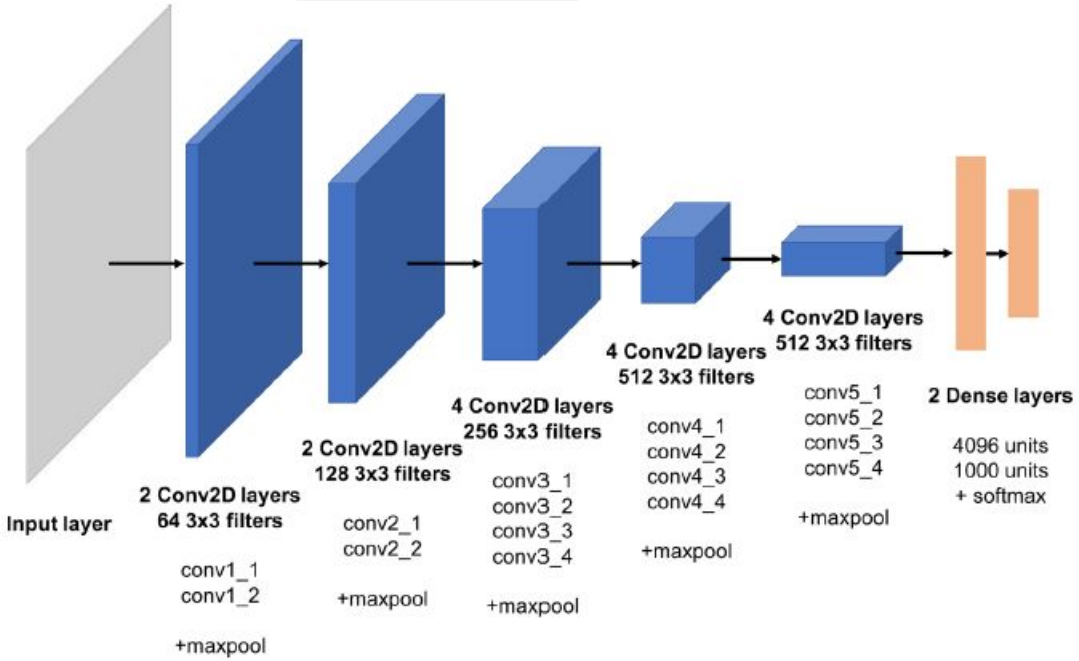
Then:

- Feed the **original image** into the network
- Feed the **generated image** into the same network
- Look at outputs from a deep layer
- Those outputs are called **feature maps**.

These feature maps represent:

- what objects exist
- where they roughly are

Content Loss



Style Loss

Content loss asks:

- "Are the objects and scene still the same?"

Style loss asks:

"Does the image *feel* like the style image?"

- For example:
- Content image → your photograph
- Style image → Van Gogh painting

After style transfer:

- your buildings and people should remain
- but the image should *look painted like Van Gogh*
- That "painting feel" is what style loss measures.

Style Loss

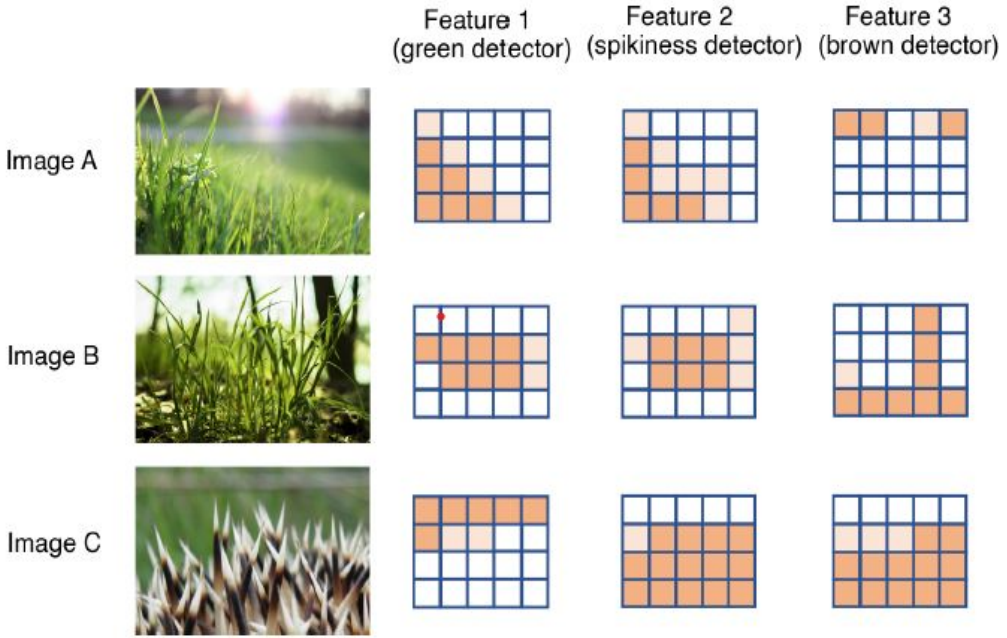
What is “style” in an image?

- Style is things like:
- textures
- brush strokes
- repeating patterns
- color combinations
- roughness/smoothness

Examples:

- grassy texture
- watercolor look
- pencil sketch appearance
- oil painting strokes

Style Loss



We measure:
"How strongly do feature maps activate together?"
If:

- green detector
- spiky detector

both become active together often,
their correlation is high

Style Loss

We measure:

“How strongly do feature maps activate together?”

If:

- green detector
- spiky detector

both become active together often,

their correlation is high

Style Loss

- **Gram Matrix**

Now we calculate this for **every pair of feature maps**.

- That huge table is called the **Gram Matrix**.
- Example:

	Green	Spiky	Brown
Green	High	High	Low
Spiky	High	High	Medium
Brown	Low	Medium	High

Similar Gram matrix = similar style

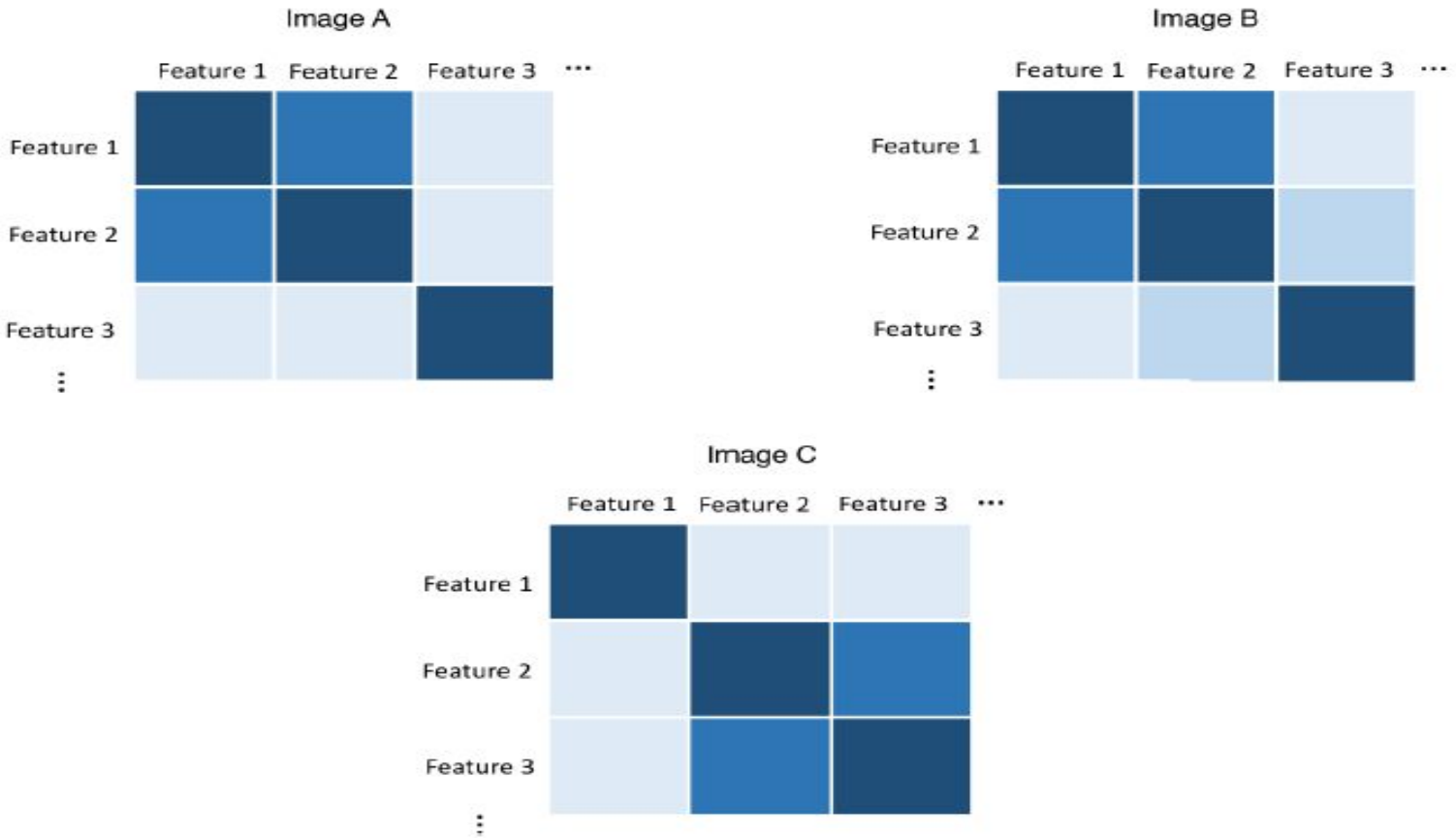


Image A
Feature 1
Feature 2
Feature 3
Feature 1
Feature 2
Feature 3
Image B
Feature 1
Feature 2
Feature 3
Feature 1
Feature 2
Feature 3
Image C
Feature 1
Feature 2
Feature 3
Feature 1
Feature 2
Feature 3

Style Loss

Style Loss Formula

- Style loss compares:
- Gram matrix of style image
- Gram matrix of generated image
- using squared error

$$L_{GM}(S, G, l) = \frac{1}{4N_l^2 M_l^2} \sum_{ij} (GM^{[l]}(S)_{ij} - GM^{[l]}(G)_{ij})^2$$

Total Variation Loss

- “Remove unwanted noise and make the image smoother.”
- During neural style transfer, the generated image can become:
- grainy
- noisy
- pixelated
- full of tiny random color changes
- Even if:
- content loss is good
- style loss is good
- the image may still look rough or unnatural.